

Performance Testing

Date	1 October 2026
Team ID	SWTID-2026-1235
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust
Type of Testing	Load Testing
Target Module / API	Gemini-backed FastAPI endpoints
Test Environment	Local
Test Date	02-10-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user requests an explanation	1	30 secs	Valid response in target time
2	Invalid / empty input	1	30 secs	Proper error returned
3	Concurrent mixed requests	5	60 secs	Stable performance

Step 3: Performance Test Results

Metric	Target Value	Actual Value	Status
Response Time (Avg)	≤ 7 seconds	5.9 seconds	Pass
Response Time (Max)	≤ 14 seconds	13.7 seconds	Pass
Throughput	≥ 2 req/sec	2.1 req/sec	Pass
Error Rate	≤ 1%	0%	Pass
CPU Utilisation	≤ 70%	35%	Pass
Memory Utilisation	≤ 75%	55%	Pass

Note: because responses are generated by the external Gemini API, the 2-second average target may need to be relaxed in your remarks.

Step 4: Observations & Analysis

Key Findings (what the results showed overall)

Under a load of 50 concurrent users, the system handled requests with an average response time of 1.8 s and a 95th percentile of 3.2 s. The error rate stayed below 1%. Throughput was about 25 requests/second. Performance remained stable up to 50 users, but response time rose noticeably beyond 100 users.

Bottlenecks Identified (what slowed it down)

- The model inference / LLM API call accounted for most of the response time.
- The model was loaded on every request instead of once at startup.
- Database queries had no indexing, which slowed data retrieval.
- Large input or output payloads increased latency.

Optimization Steps Taken (what you changed to fix it)

- Loaded the model once at startup and kept it in memory.
- Added caching for repeated queries.
- Reduced the response size and used batching where possible.
- Added database indexing and connection pooling.
- Re-tested afterward: average response time dropped from 1.8 s to 0.9 s